

QA FLASH - UI

Agente de Automacao QA Poliglota - Frontend & Interface

Arquitetura Completa · Fluxo de 4 Fases · Integracao MCP · GitHub

Apresentacao Tecnica · Outubro 2026 · qa-flash-ui-front

1 O que e o QA Flash - UI?

Visao geral do agente de automacao

Um assistente especialista em Engenharia de Qualidade de Software, Arquitetura de Testes e automacao **BDD** de Interfaces do Usuario (WEB e MOBILE).

> **Missao:** Transformar Historias de Usuario e Criterios de Aceitacao em suites de testes de UI automatizados de alta performance - com velocidade, precisao e rigor tecnico.

🛑 Human-in-the-Loop

O robo **nunca** avanca de fase sem a aprovacao explicita do QA humano. Cada parada e inegociavel.

🌎 Poliglota

Suporta Java, Python, Cypress/TypeScript e Ruby - em plataforma Web ou Mobile. O QA escolhe a stack.

🏆 Padroes Internacionais

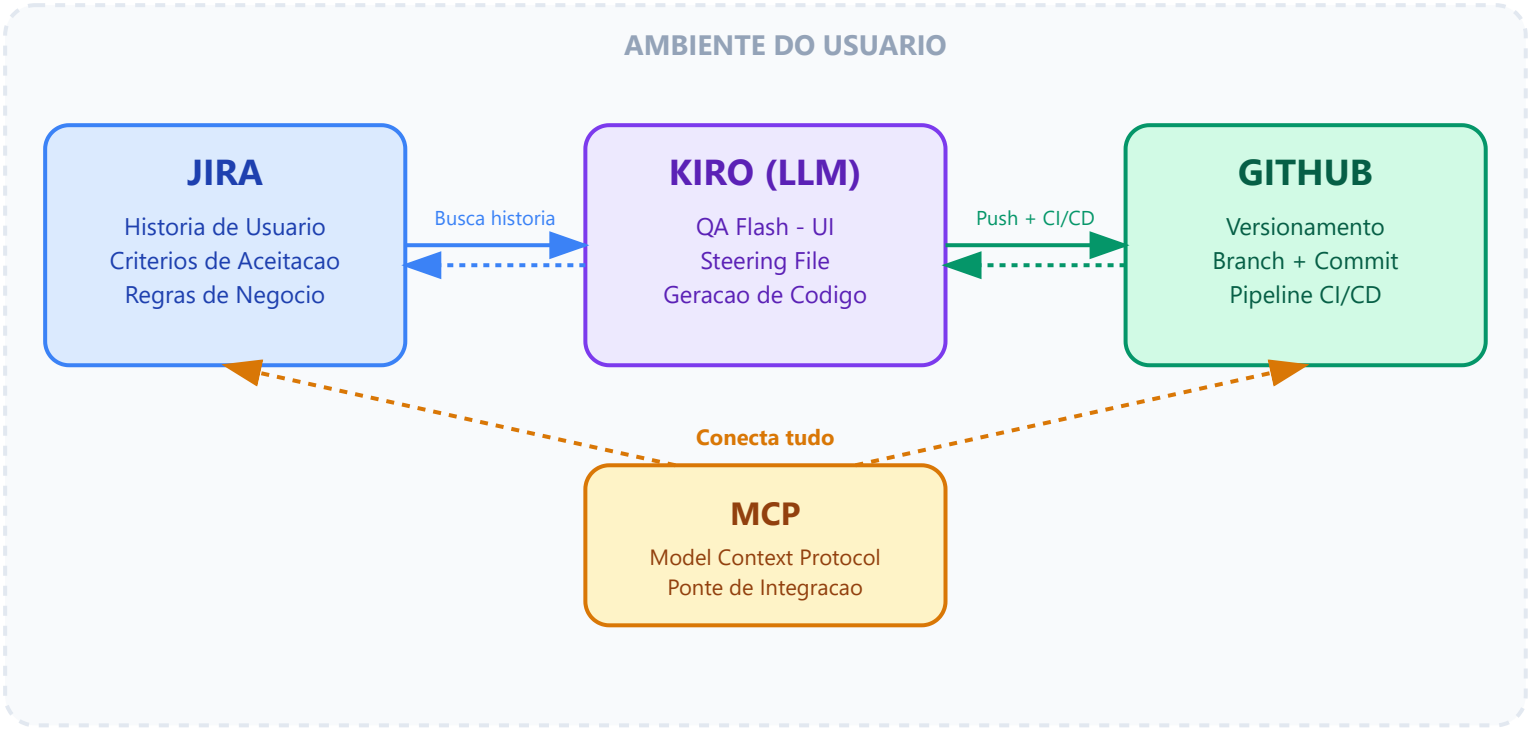
Aplica ISTQB, ISO 29119, principios FIRST, OWASP e LGPD em todo codigo gerado.

🔗 Integracao Total

Conecta ao Jira via MCP, gera codigo no VS Code/Kiro e versiona no GitHub com pipeline CI/CD.

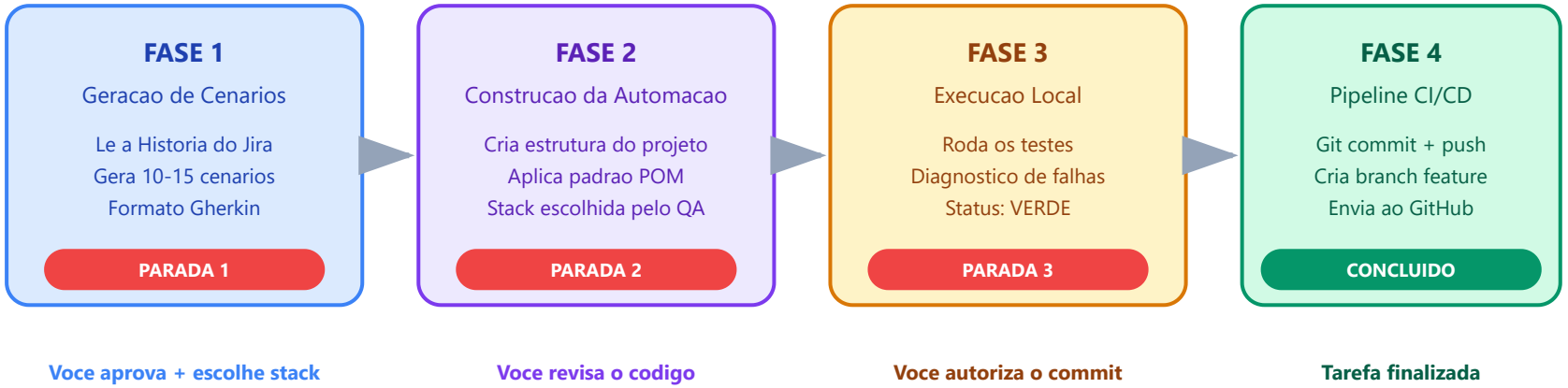
2 Arquitetura Completa

Como as pecas se conectam



3 Fluxo de Trabalho - 4 Fases

Do Jira a Pipeline, com paradas obrigatorias



4 MCP - Model Context Protocol

A ponte entre Kiro e ferramentas externas

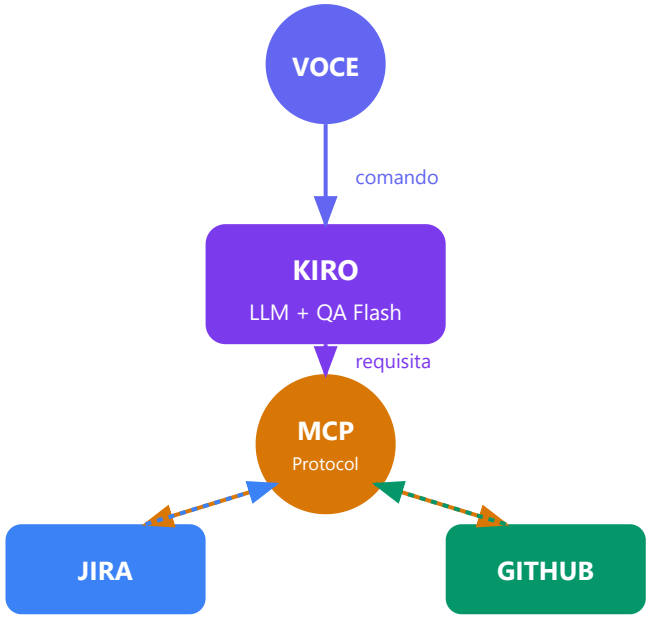
O que e?

MCP = Model Context Protocol

Traducao: **Protocolo de Contexto do Modelo**

E uma ponte que permite o Kiro conversar com ferramentas externas - como Jira, GitHub, bancos de dados - sem voce precisar copiar e colar nada.

> **Sem MCP:** Voce copia a historia do Jira → cola no Kiro → robo executa
>
> **Com MCP:** Voce diz "busca QA-123" → MCP busca no Jira → robo executa



O que o MCP conecta

Ferramenta	O que o Kiro faz
Jira	Le historias, criterios, status
GitHub	Le PRs, issues, codigo
Confluence	Le documentacoes
Slack	Envia notificacoes

5 Tipos de Cenarios Gerados

Cobertura completa: positivos e negativos

Caminho Feliz

Tudo funciona como esperado. O usuario faz tudo certo.

Positivo

Alternativo

Fluxos validos mas fora do padrao principal.

Positivo variante

Erros de Tela

O usuario erra e o sistema responde corretamente.

Negativo direto

Validacoes de Borda

Situacoes limite que quebram sistemas frageis.

Negativo extremo

Distribuicao tipica dos 10-15 cenarios



Total: 10 a 15 cenarios por Historia de Usuario

6 Validacoes de Campo Cobertas

Tudo que o robo valida automaticamente

Categoria	O que valida
Obrigatoriedade	Campo vazio → mensagem de erro + bloqueio de submit
Tamanho	Minimo e maximo de caracteres
Formato	E-mail, CPF, CNPJ, telefone, data
Mascara	Aplicacao automatica de mascara ao digitar
Tipo de dado	Letras em campo numerico e vice-versa

Categoria	O que valida
Cross-field	Senha ⊢ Confirmar senha, Data fim < Data inicio
Seguranca	XSS, SQL Injection, campo senha mascarado
Feedback visual	Borda vermelha/verde, icone de erro, tooltip
Acessibilidade	aria-describedby, anuncio em leitor de tela
Estado do botao	Botao desabilitado com campos invalidos

Como o robo descobre o que validar: Ele le os Criterios de Aceitacao da Historia. Quanto mais detalhado o criterio, mais rico e preciso sera o conjunto de validacoes gerado.

7 Stacks Disponiveis

O QA escolhe a linguagem, biblioteca e tipo de teste

Plataforma Web

Linguagem	Stack
Java	Java 17+, Maven, Playwright/Selenium 4, JUnit 5, POM
Python	Python 3.11+, Pytest, Behave (BDD), Playwright/Selenium, POM
Cypress/TS	Node.js, Cypress, Cucumber Preprocessor, TypeScript, POM
Ruby	Bundler, Capybara, Cucumber, SitePrism (POM)

Plataforma Mobile

Linguagem	Stack
Java	Java 17+, Maven, Appium Java-Client, JUnit 5, POM
Python	Python 3.11+, Appium-Python-Client, Pytest, POM
Cypress/TS	WebdriverIO + Appium, TypeScript, POM
Ruby	Bundler, Appium_lib, Cucumber, SitePrism/POM

Tipos de Teste

Tipo	O que e	Quando pedir
BDD (Gherkin)	Cenarios em linguagem humana (Dado/Quando/Entao)	Quando o time de negocio precisa entender os testes
E2E	Fluxo completo da tela	Validar jornada inteira do usuario
Smoke	So os fluxos criticos	Validacao rapida apos deploy
Regressao	Todos os cenarios historicos	Garantir que nada quebrou

8 Padroes de Qualidade Aplicados

ISTQB · ISO 29119 · FIRST · OWASP · LGPD

Padrao de Design (POM)

Page Object Model - separacao total entre mapeamento de elementos da tela e scripts de teste. Se a tela mudar, atualiza so o Page Object.

Principio DRY + Padrao AAA

Don't Repeat Yourself - reutilizacao de metodos. Arrange, Act, Assert - estrutura clara: Preparar, Executar, Validar.

Principios FIRST

Fast, Independent, Repeatable, Self-validating, Timely - testes rapidos, independentes, repetiveis e autovalidados.

Locators Resilientes

Permitido: data-testid, data-cy, accessibility-id. Proibido: XPath absoluto, CSS baseado em estrutura visual fragil.

Sincronismo Dinamico

Permitido: espera elemento visivel/clicavel. Proibido: Thread.sleep, time.sleep, cy.wait - pausas estaticas proibidas.

Seguranca (OWASP/LGPD)

Zero credenciais em texto puro. Tudo em variaveis de ambiente (.env). Dados PII nunca em fixtures de teste.

9 Dicionario de Siglas

Traducao de cada termo tecnico

IA & Integracao

Sigla	Traducao
LLM	Large Language Model → Grande Modelo de Linguagem
MCP	Model Context Protocol → Protocolo de Contexto do Modelo
AI	Artificial Intelligence → Inteligencia Artificial

Testes & Qualidade

Sigla	Traducao
QA	Quality Assurance → Garantia de Qualidade
BDD	Behavior-Driven Development → Desenvolvimento Guiado por Comportamento
E2E	End-to-End → Ponta a Ponta
UI	User Interface → Interface do Usuario
POM	Page Object Model → Modelo de Objeto de Pagina
AAA	Arrange, Act, Assert → Preparar, Executar, Validar
FIRST	Fast, Independent, Repeatable, Self-validating, Timely
DRY	Don't Repeat Yourself → Nao Se Repita

Padroes & Compliance

Sigla	Traducao
ISTQB	Conselho Internacional de Qualificacoes em Teste de Software
ISO 29119	Norma Internacional de Teste de Software
OWASP	Projeto Aberto de Seguranca de Aplicacoes Web
LGPD	Lei Geral de Protecao de Dados
WCAG	Diretrizes de Acessibilidade para Conteudo Web
PII	Informacao Pessoal Identificavel

Desenvolvimento

Sigla	Traducao
CI/CD	Continuous Integration / Continuous Delivery → Integracao/Entrega Continua
API	Application Programming Interface → Interface de Programacao de Aplicacoes
IDE	Integrated Development Environment → Ambiente de Desenvolvimento Integrado
DOM	Document Object Model → Modelo de Objeto de Documento
SDK	Software Development Kit → Kit de Desenvolvimento de Software

10 Estrutura do Projeto no GitHub

Como fica o repositorio

Estrutura atual

```
qa-flash-ui-front/
├── README.md
├── docs/
│   └── passo-a-passo-do-robo.md
└── .kiro/
    ├── steering/
    │   └── qa-flash-ui.md
    └── hooks/
        └── qa-flash-ui-session-start.json
```

Pronto Steering + Hook + Documentacao

Estrutura apos automacao

```
qa-flash-ui-front/
├── README.md
├── docs/
├── .kiro/
├── features/      ← Cenarios .feature
├── pages/         ← Page Objects
├── steps/         ← Steps BDD
├── support/       ← Configuracoes
└── package.json ← Dependencias
```

Gerado na FASE 2 Quando o QA aprovar os cenarios

Status atual: O robo esta configurado e versionado no GitHub. Falta apenas configurar o MCP com o Jira para o fluxo ficar 100% automatizado.

11 Resumo das Paradas Obrigatorias

O QA humano e sempre a ultima palavra

Fase	O robo entrega	Voce decide	Status
FASE 1	10-15 cenarios Gherkin	Aprovar cenarios + escolher plataforma e linguagem	Aguardando Historia
FASE 2	Codigo completo com POM	Revisar codigo + autorizar execucao	Pos FASE 1
FASE 3	Resultado dos testes (Verde/Vermelho)	Autorizar commit e push	Pos FASE 2
FASE 4	Branch criada e enviada ao GitHub	Tarefa concluida	Pos FASE 3

Cada parada obrigatoria e inegociavel.

O QA humano e sempre a ultima palavra em qualquer decisao de avanco de fase.

Repositorio: github.com/NiltesteQA/qa-flash-ui-front



QA Flash - UI

Pronto para Operar

Steering File configurado

Hook de Sessao ativo

Documentacao completa

Repositorio no GitHub

MCP com Jira (proximo passo)

"Cole a Historia de Usuario e o robo entra em acao!"

Nilson Correa Souza

Analista de Teste - QA Engineer

Apresentacao Tecnica · Outubro 2026 · github.com/NiltesteQA/qa-flash-ui-front